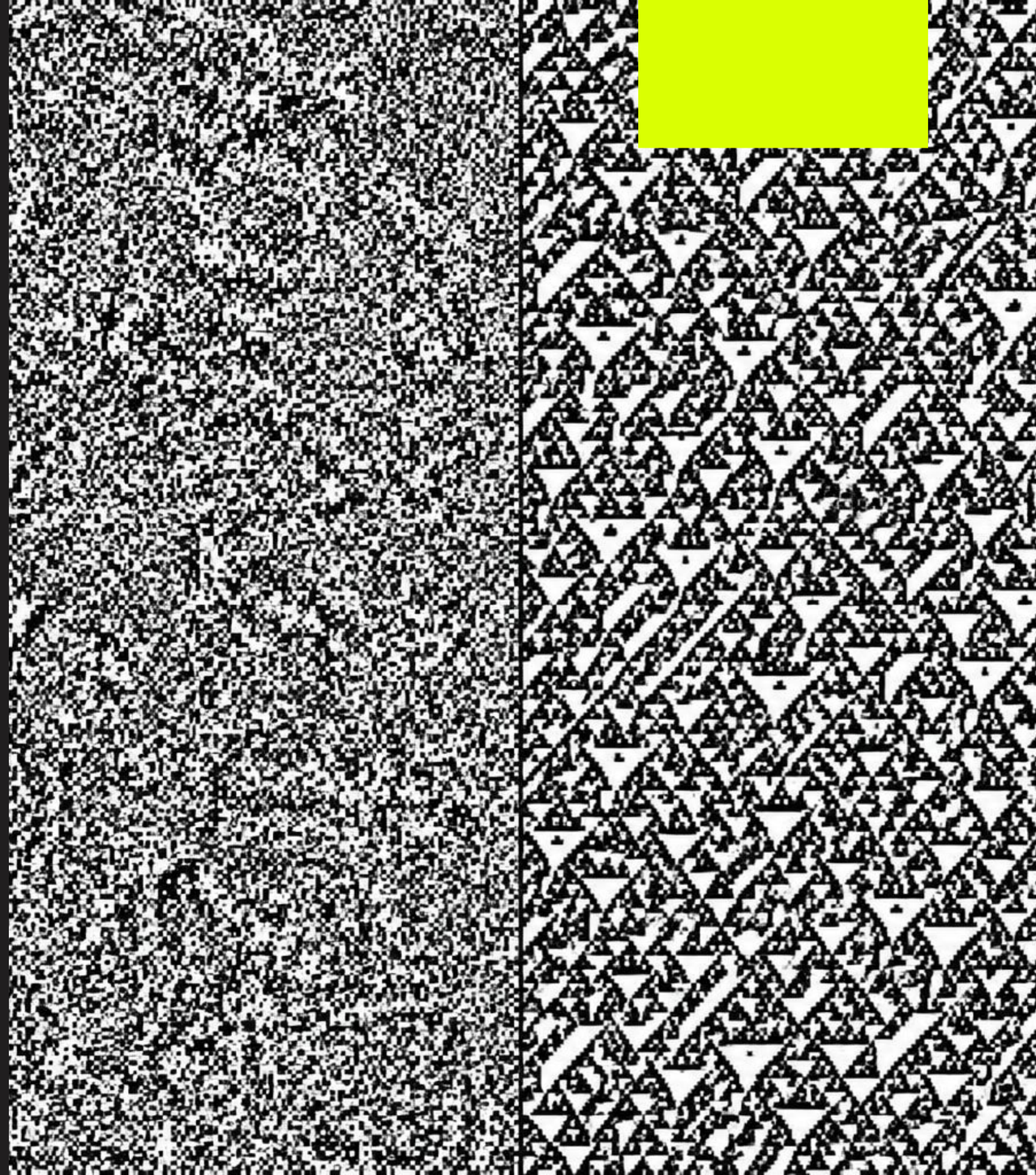
Complexity-Guide d *Pre-Pretraining*

Can We Make Language Models Better by Teaching Them Baby Puzzles First?

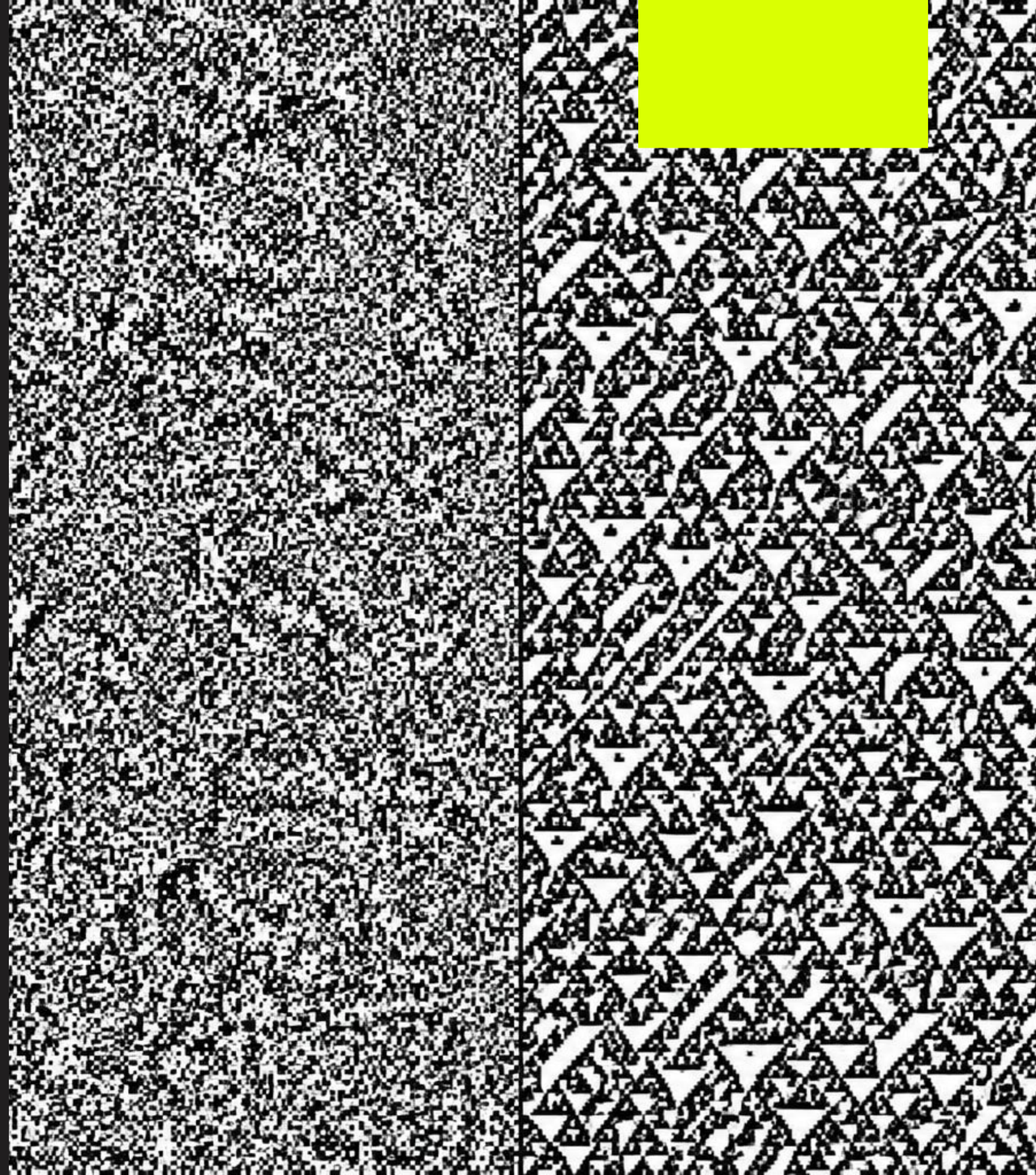
Nicholas Kluge Corrêa
Postdoc @ Bonn-Aachen International Center for Information Technology (b-it)



b-it Bonn-Aachen International Center for Information Technology

UNIVERSITÄT BONN

What if, before you ever taught a language model to read English, you first taught it to recognize simple patterns — like ABABAB or ABCCBA — Would that make it better at English later?



Our Roadmap for the next ~30 min ...

Why would you do this?

>> I'll (try to) convince you that “pre-pretraining” is an idea worth thinking about.

What does complexity even mean?

>> Present some tools that can be used to categorize data.

How do we run the experiments?

>> I'll show you what are the kind of experiments we are running?

What did we find?

>> Next presentation? Or skip straight to the current results (SPOILERS)

Why would you do this?

Because babies don't start with Tolstoy. They start with peek-a-boo! And there is some supporting evidence to support the idea we should approach LLMs in a similar fashion …

>> Program synthesis improves with synthetic pre-training … (1509.00413)
>> Math reasoning benefits from formal language exposure … (1904.01557)
>> Models trained without ANY human language still develop linguistic capabilities … (2012.11995)
>> Pre-training on formal languages imparts measurable linguistic biases … (2304.13060)

Learning simple structure first makes models better at learning complex structure later …

TRADITIONAL PIPELINE

Pretrain (LM)

Pre-training on large text corpus

Natural language corpus

Mid-train

(e.g., further domain-specific training or intermediate steps)

Post-train (SFT)

Supervised Fine-Tuning

Fine-Tuning for specific tasks

WITH PRE-PRETRAINING

Pre-pretrai-ning

(synthetic data)

Pretrain (LM)

Natural language data

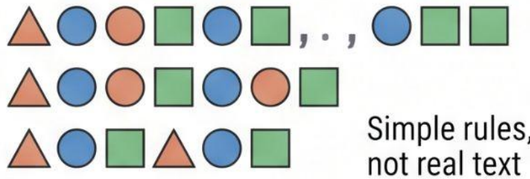
Mid-train

Post-train (SFT)

Supervised Fine-Tuning for specific tasks

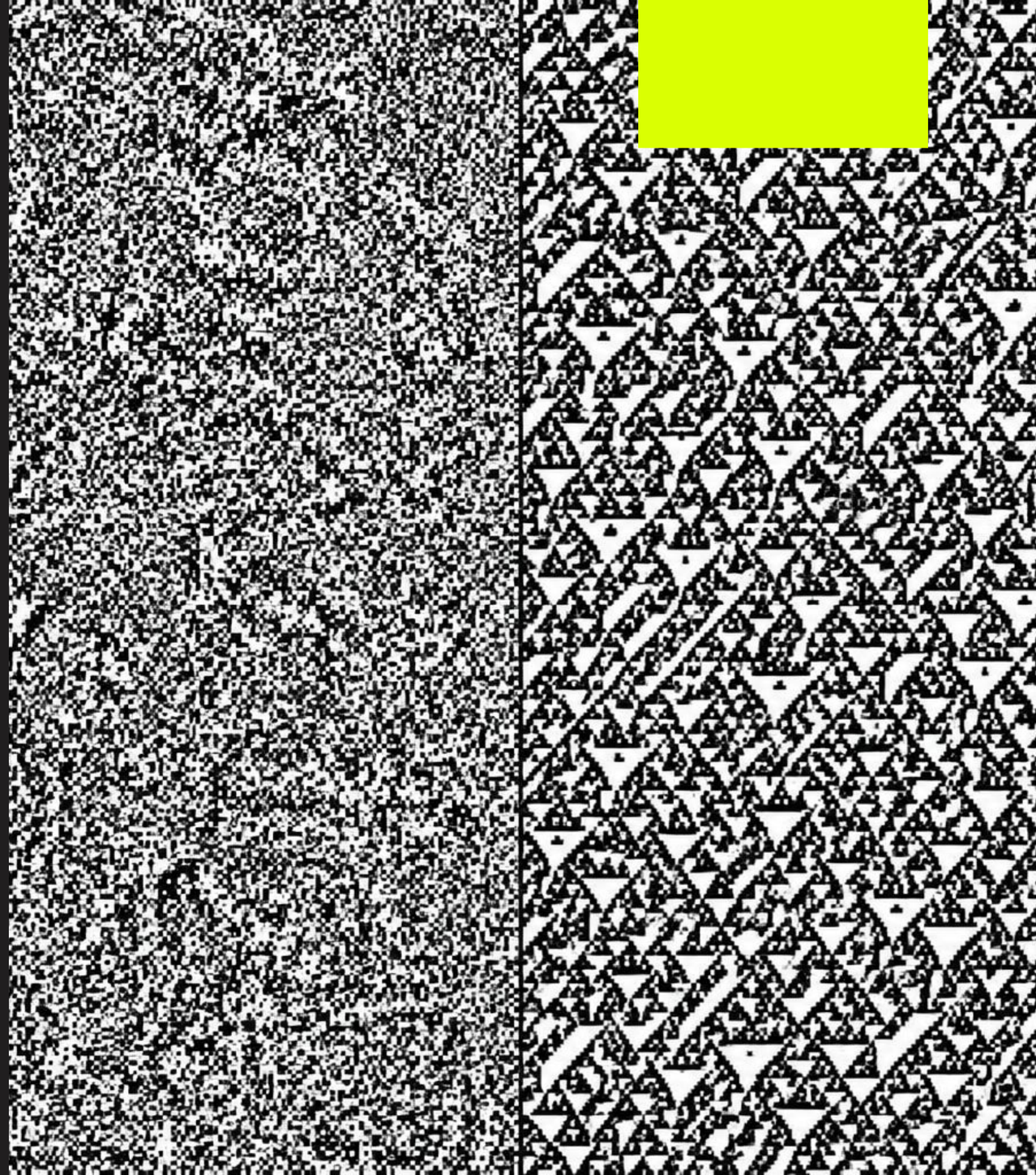
Model sees patterns first, real language second.

Synthetic, rule-generated data



Simple rules, not real text

Can we make this systematic? Can we measure the structure in the data, and use those measurements to predict what will transfer best?



What does complexity even mean?

Algorithmic Information Theory (AIT) concerns itself with the relationship between computation and information.

```
A:   "01010101010101010101010101010101..."
```

What does complexity even mean?

Algorithmic Information Theory (AIT) concerns itself with the relationship between computation and information.

```
A:  "01010101010101010101010101010101..."  ←  print("01" * 32)
```

What does complexity even mean?

Algorithmic Information Theory (AIT) concerns itself with the relationship between computation and information.

```
A:  "01010101010101010101010101010101..."  ←  print("01" * 32)

B:  "11001000011000011101111011101100..."
```

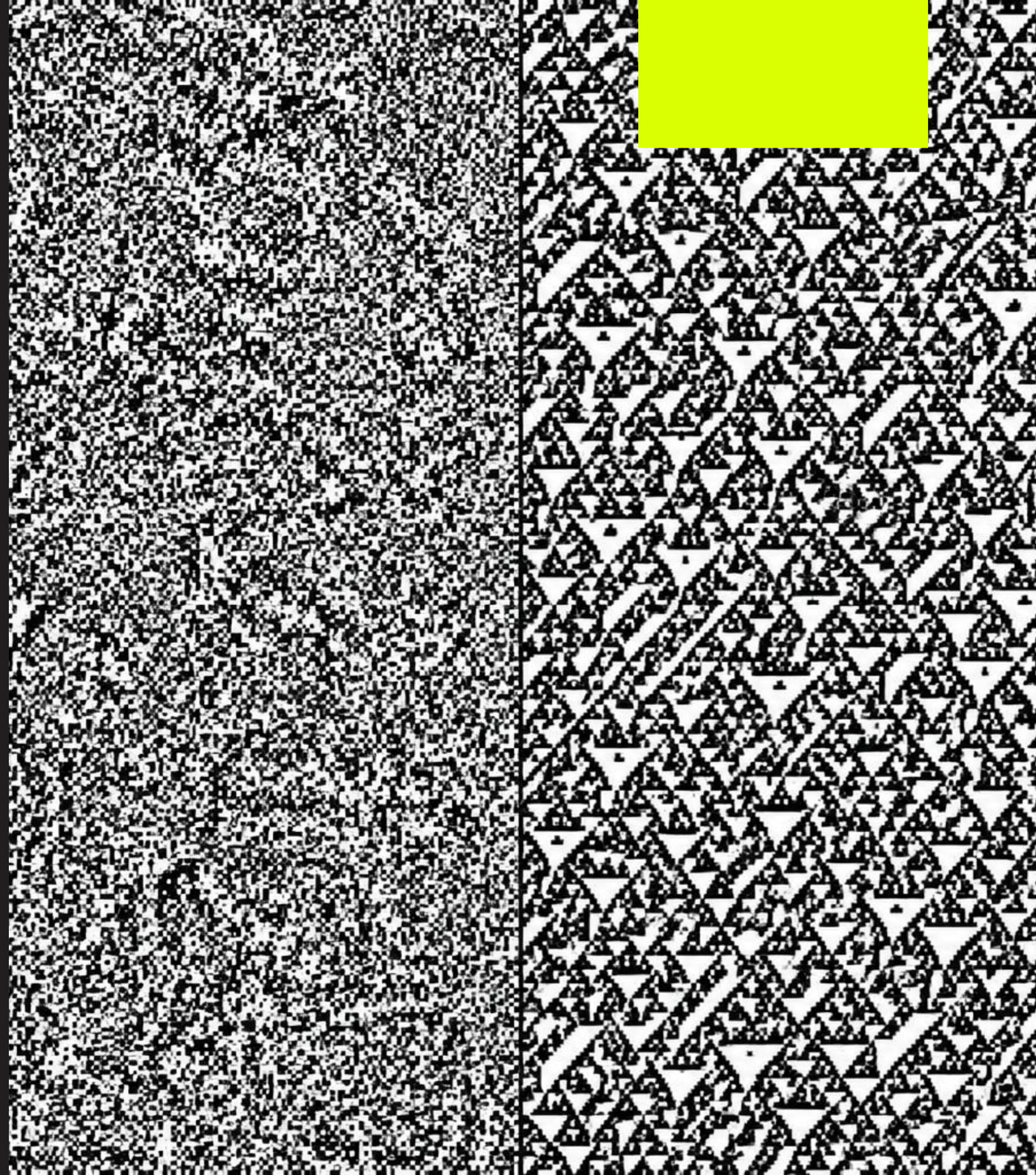
What does complexity even mean?

Algorithmic Information Theory (AIT) concerns itself with the relationship between computation and information.

```
A:  "01010101010101010101010101010101..."  ←  print("01" * 32)

B:  "11001000011000011101111011101100..." ←  print("11001000...")
```

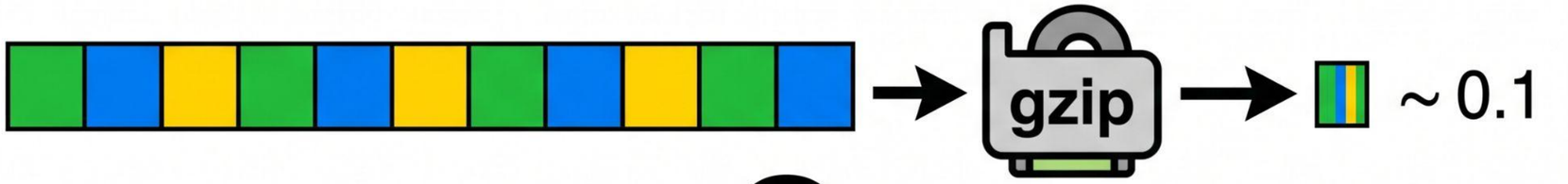
Sadly, Kolmogorov complexity is incomputable — you can't write a program that finds the shortest program for any input. So we need an approximation.



Kolmogorov complexity K(x): “Shortest program that outputs x”

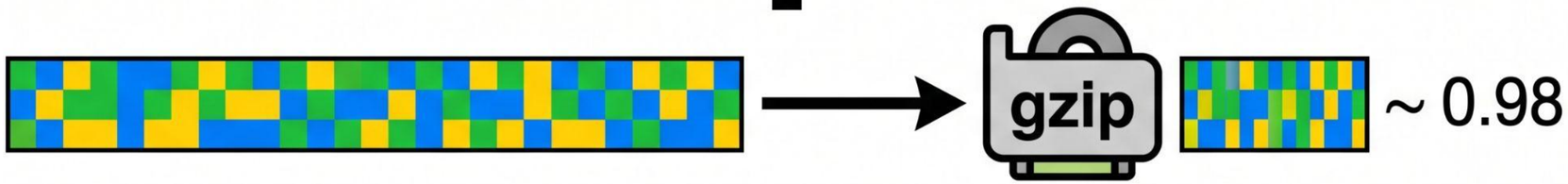
INCOMPUTABLE

gzip compression ratio: size(compressed) / size(original)



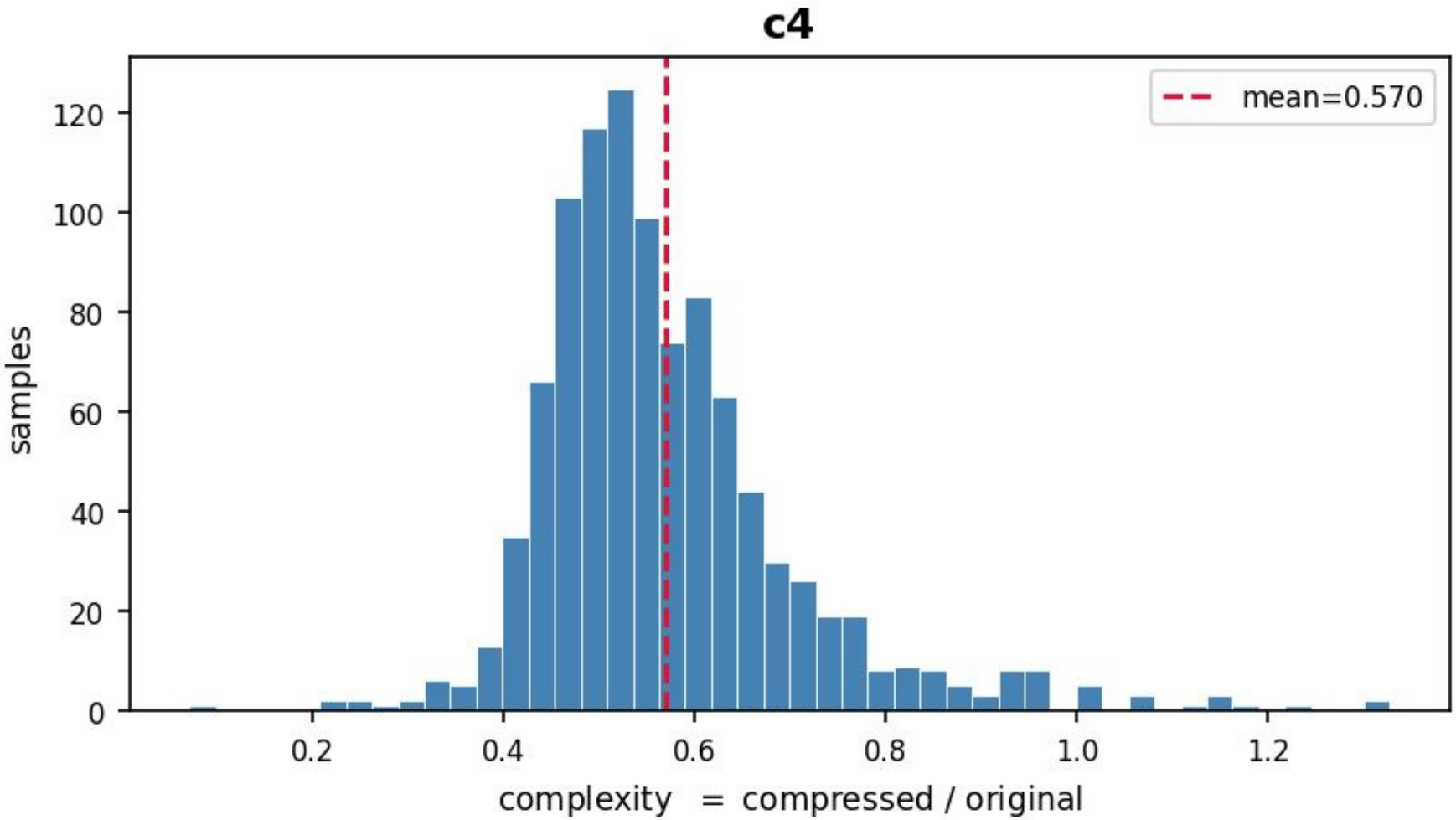
$\mathbf{K}(x)$?

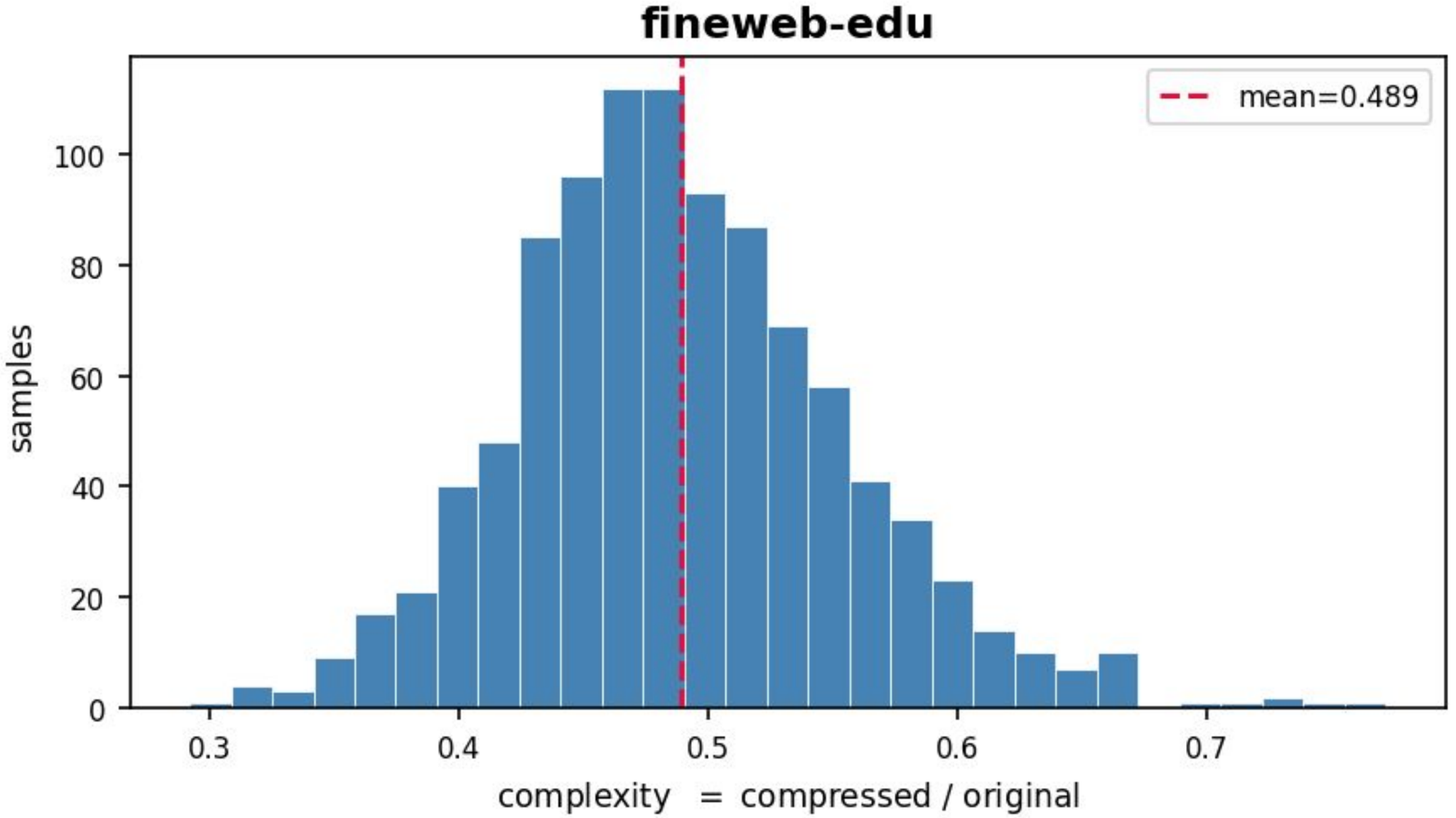
We approximate the uncomputable with something practical.

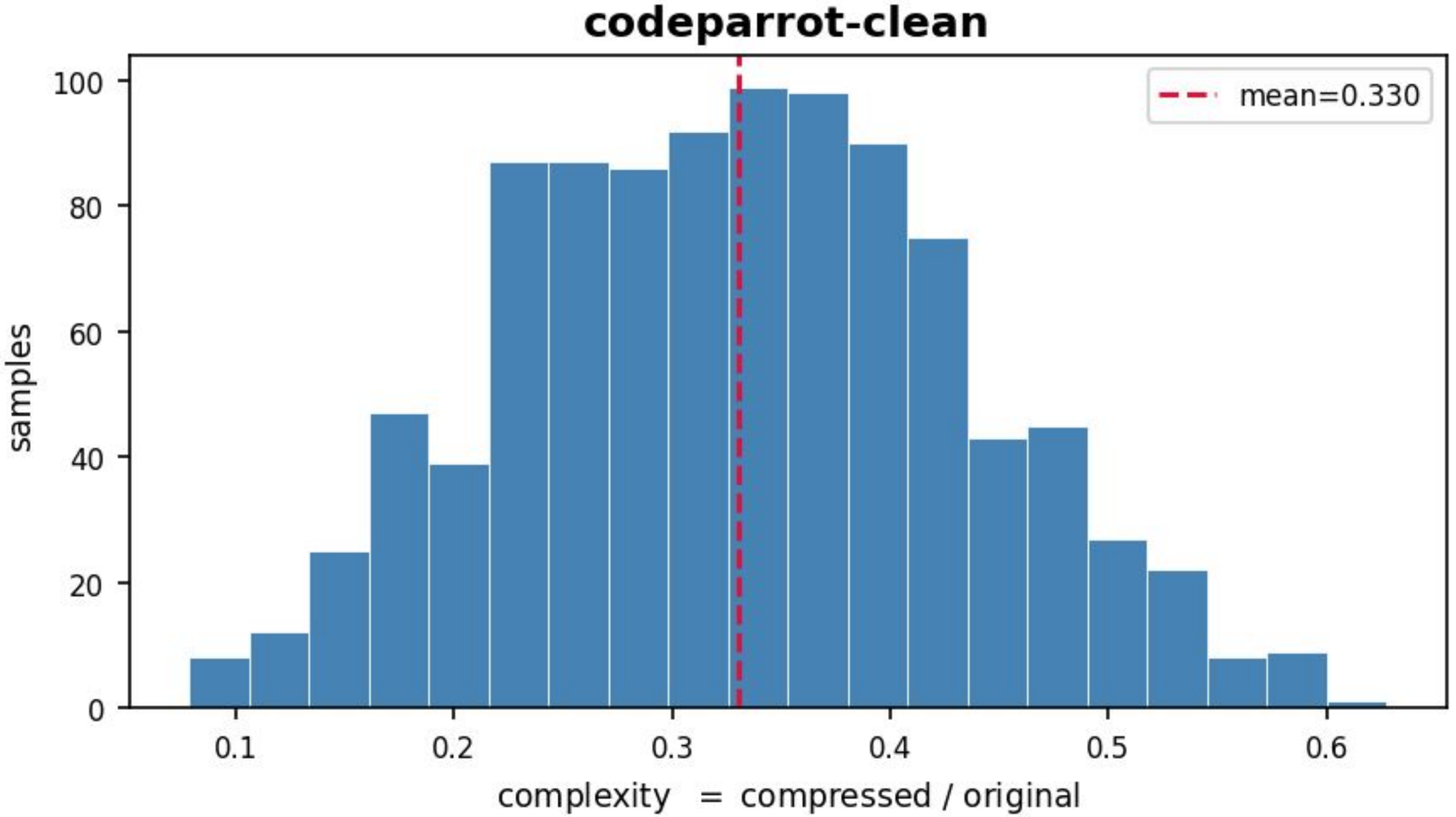


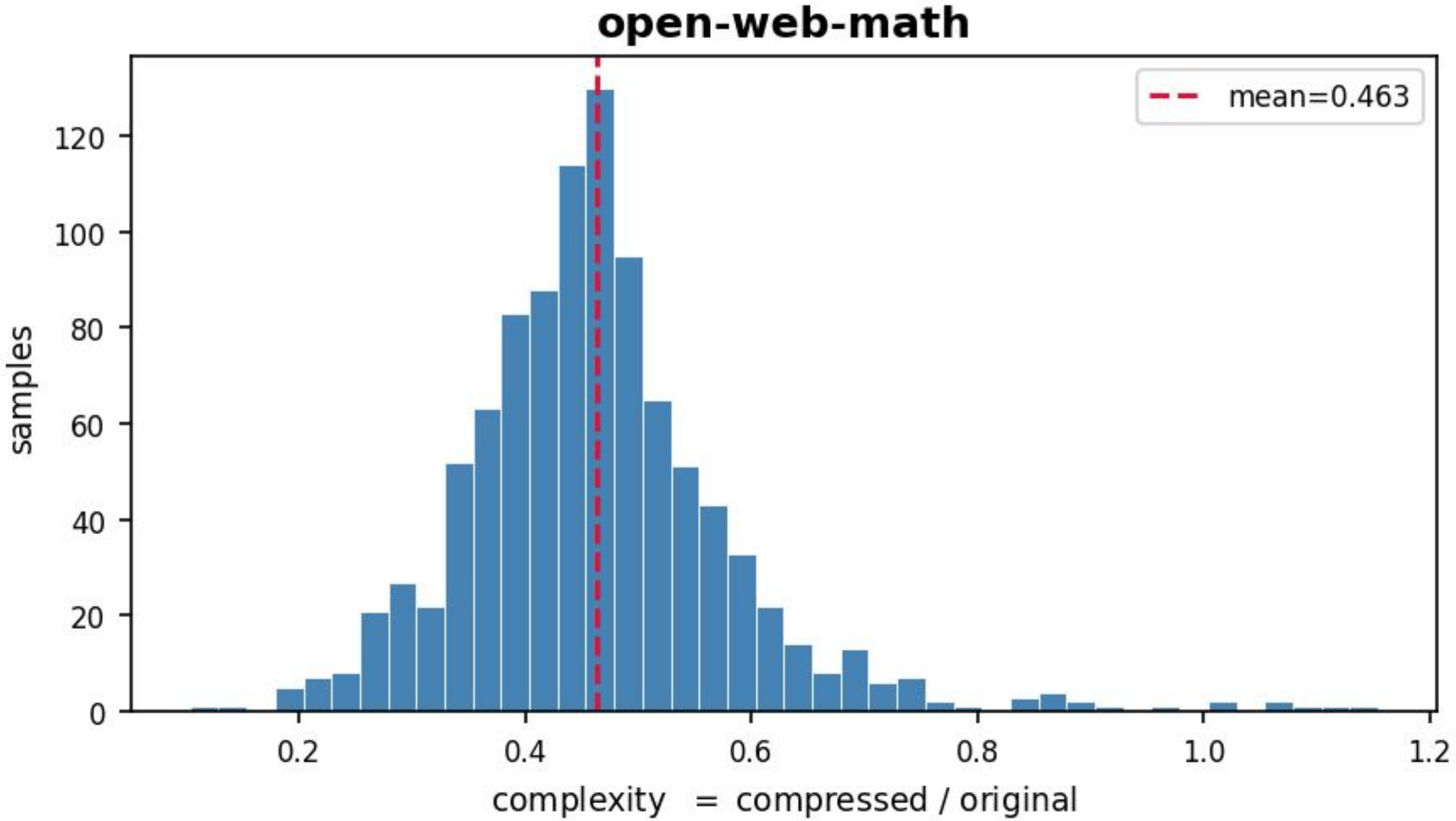
gzip ratio → 0.0 “Very compressible”

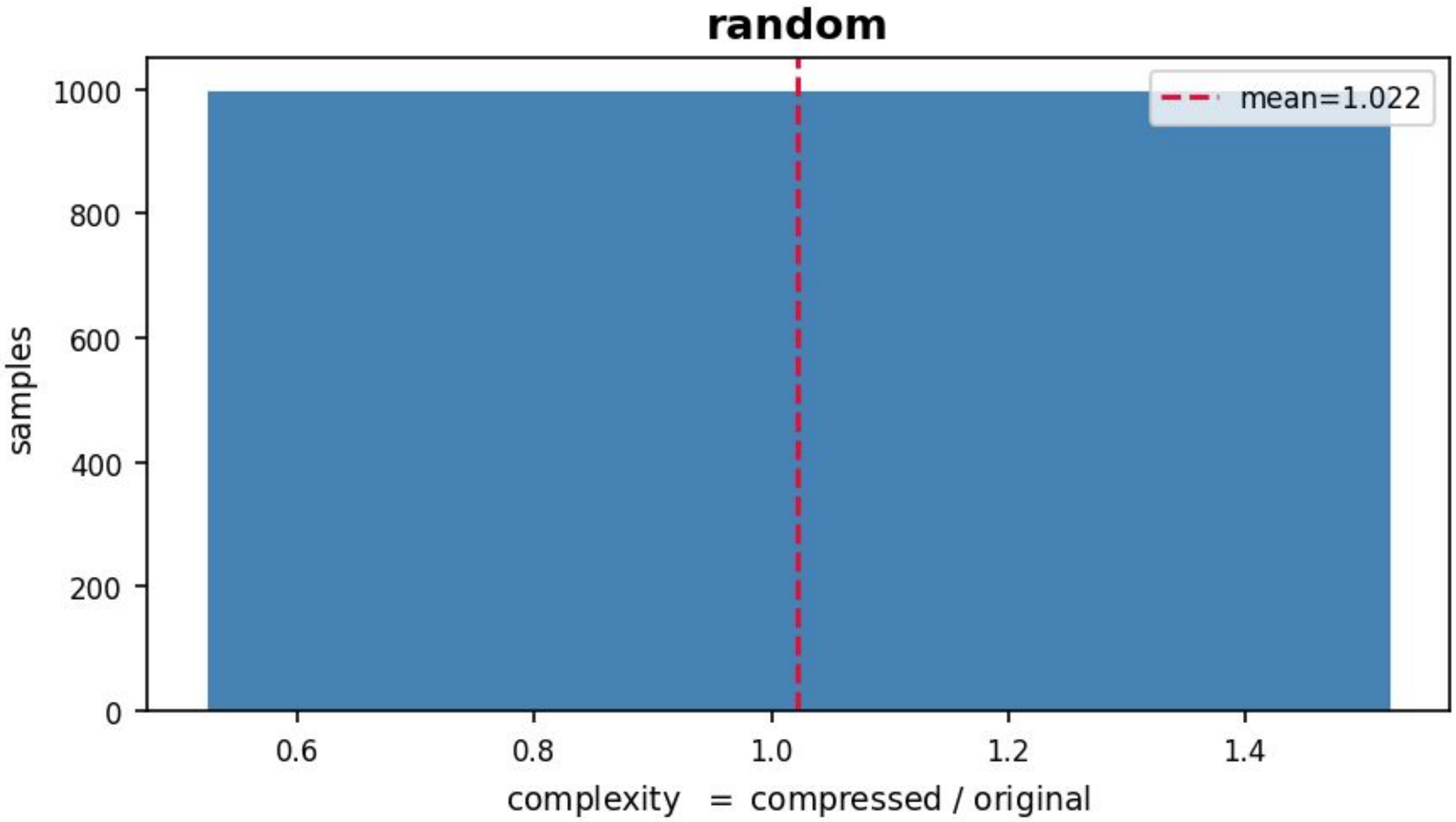
gzip ratio → 1.0 “Incompressible”

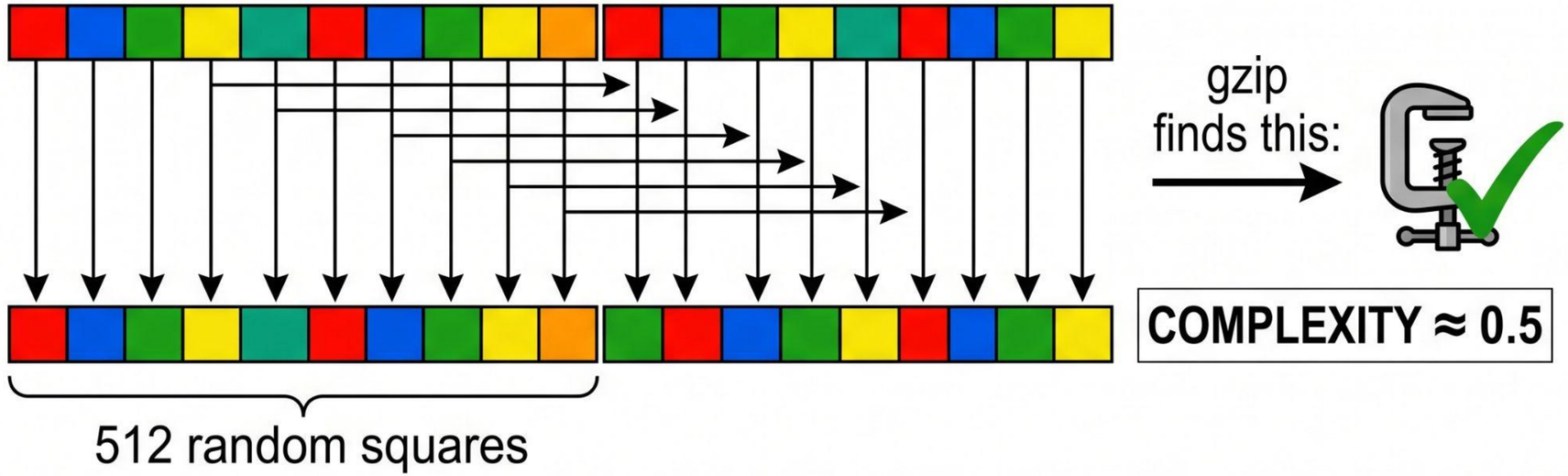

c4
mean=0.570
samples
120
100
80
60
40
20
0
0.2
0.4
0.6
0.8
1.0
1.2
complexity = compressed / original

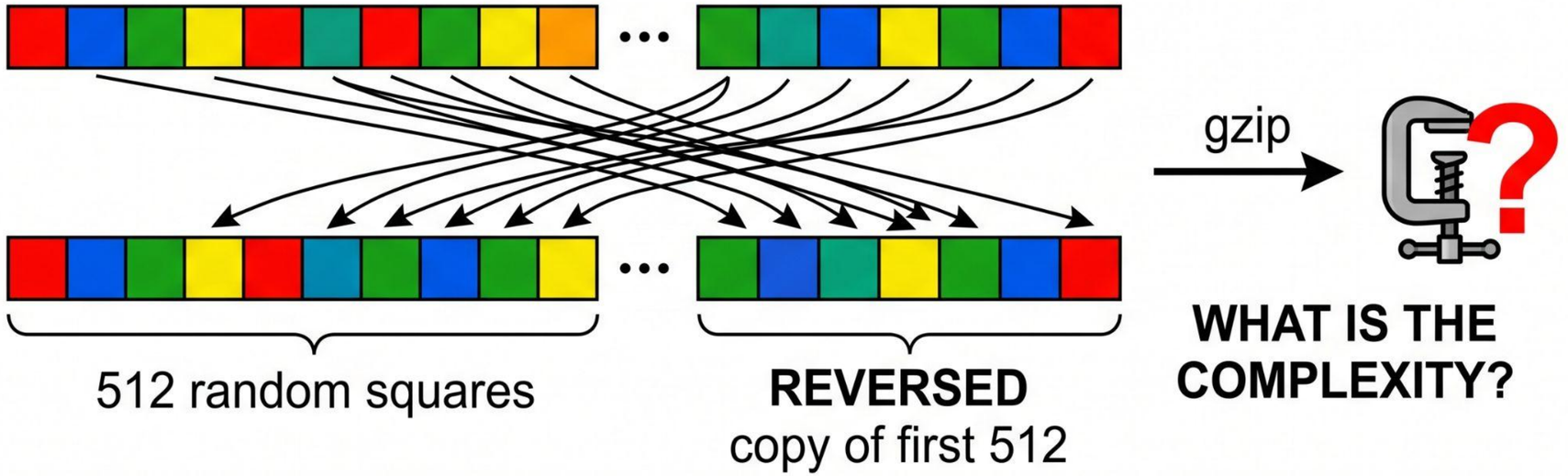

fineweb-edu
mean=0.489
samples
100
80
60
40
20
0
0.3
0.4
0.5
0.6
0.7
complexity = compressed / original


codeparrot-clean
mean=0.330
samples
100
80
60
40
20
0
0.1
0.2
0.3
0.4
0.5
0.6
complexity = compressed / original

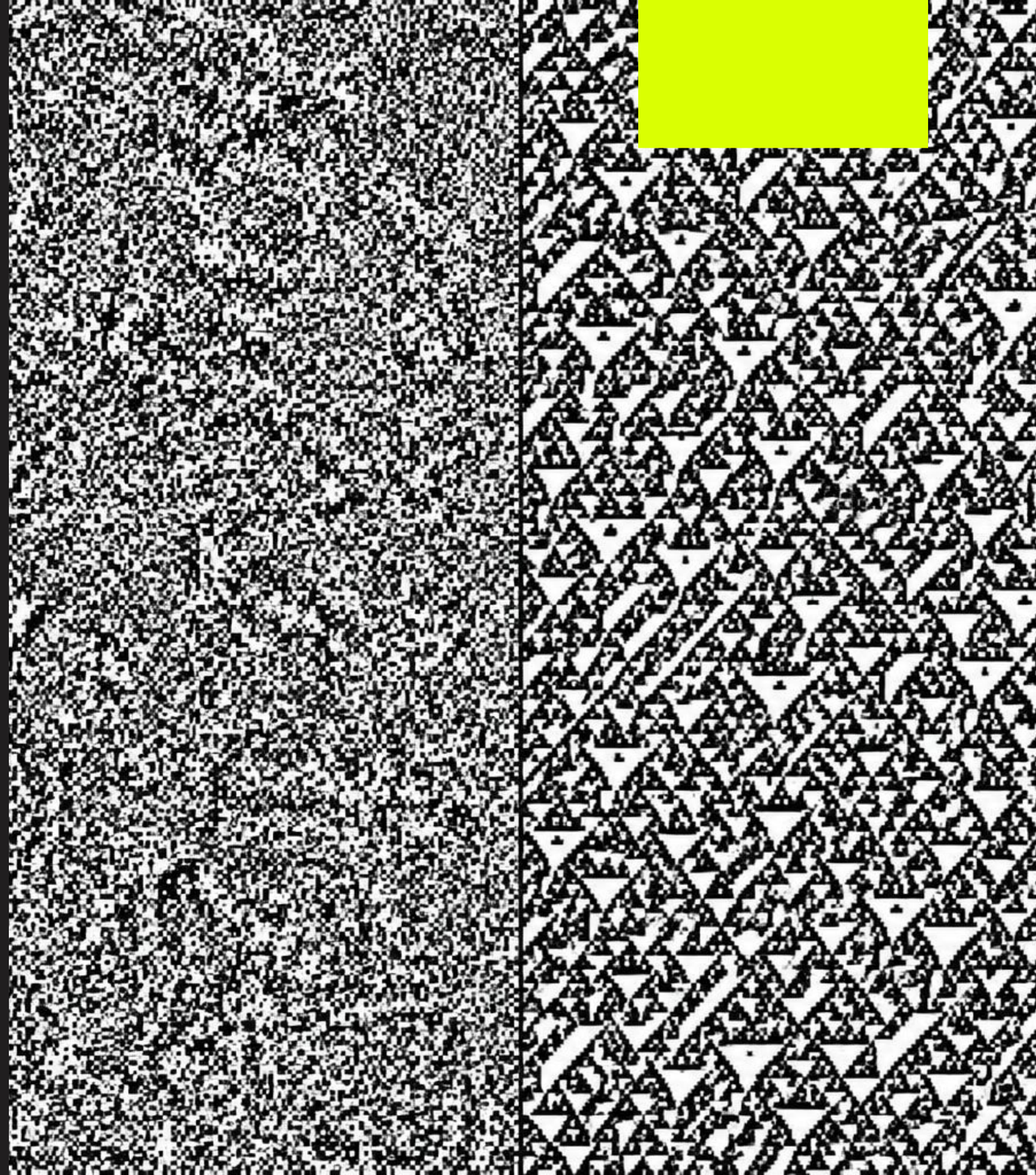

open-web-math
mean=0.463
samples
0
20
40
60
80
100
120
0.2
0.4
0.6
0.8
1.0
1.2
complexity = compressed / original

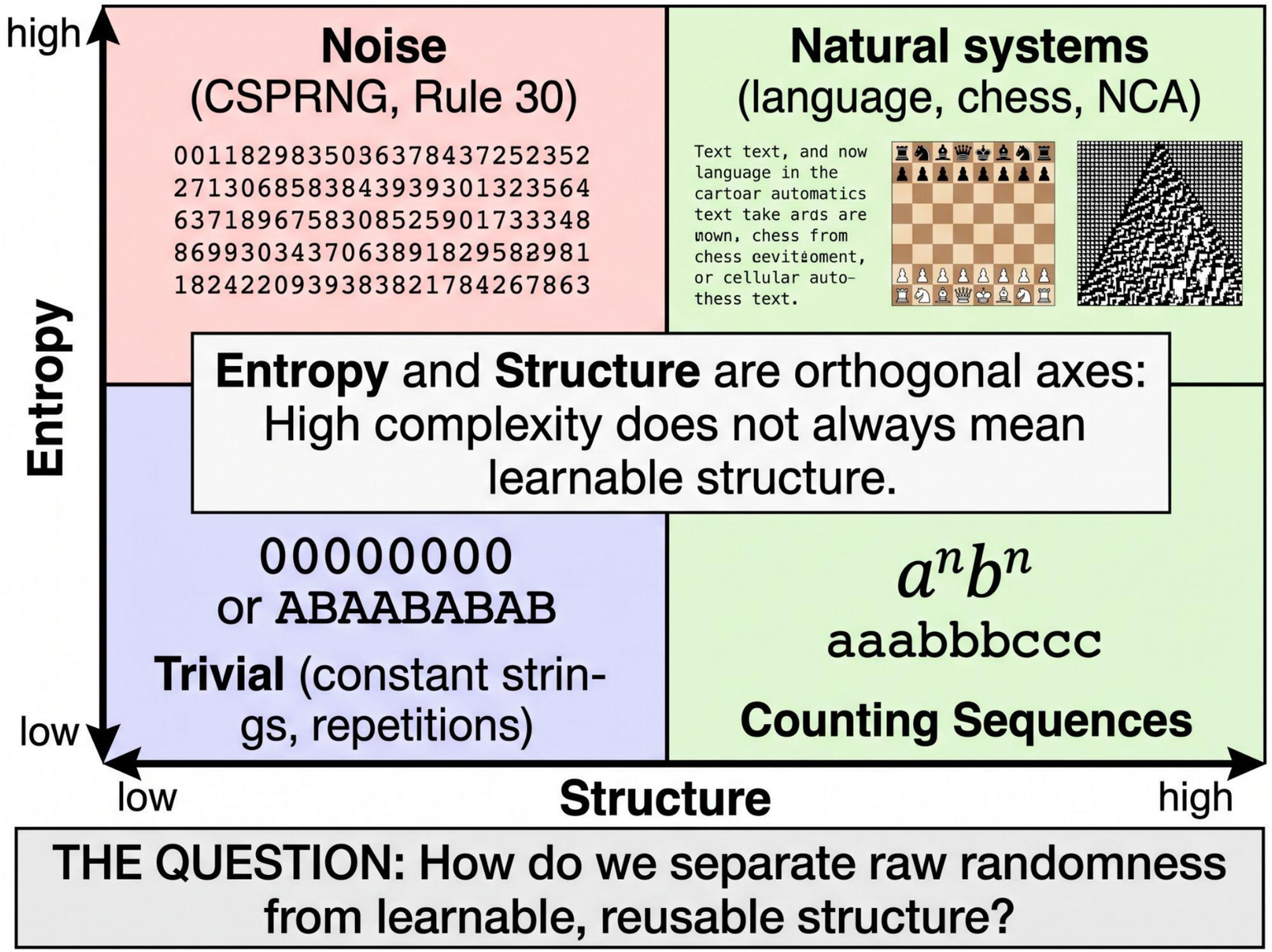

random
mean=1.022
samples
1000
800
600
400
200
0
0.6
0.8
1.0
1.2
1.4
complexity = compressed / original


gzip
finds this:
COMPLEXITY ≈ 0.5
512 random squares

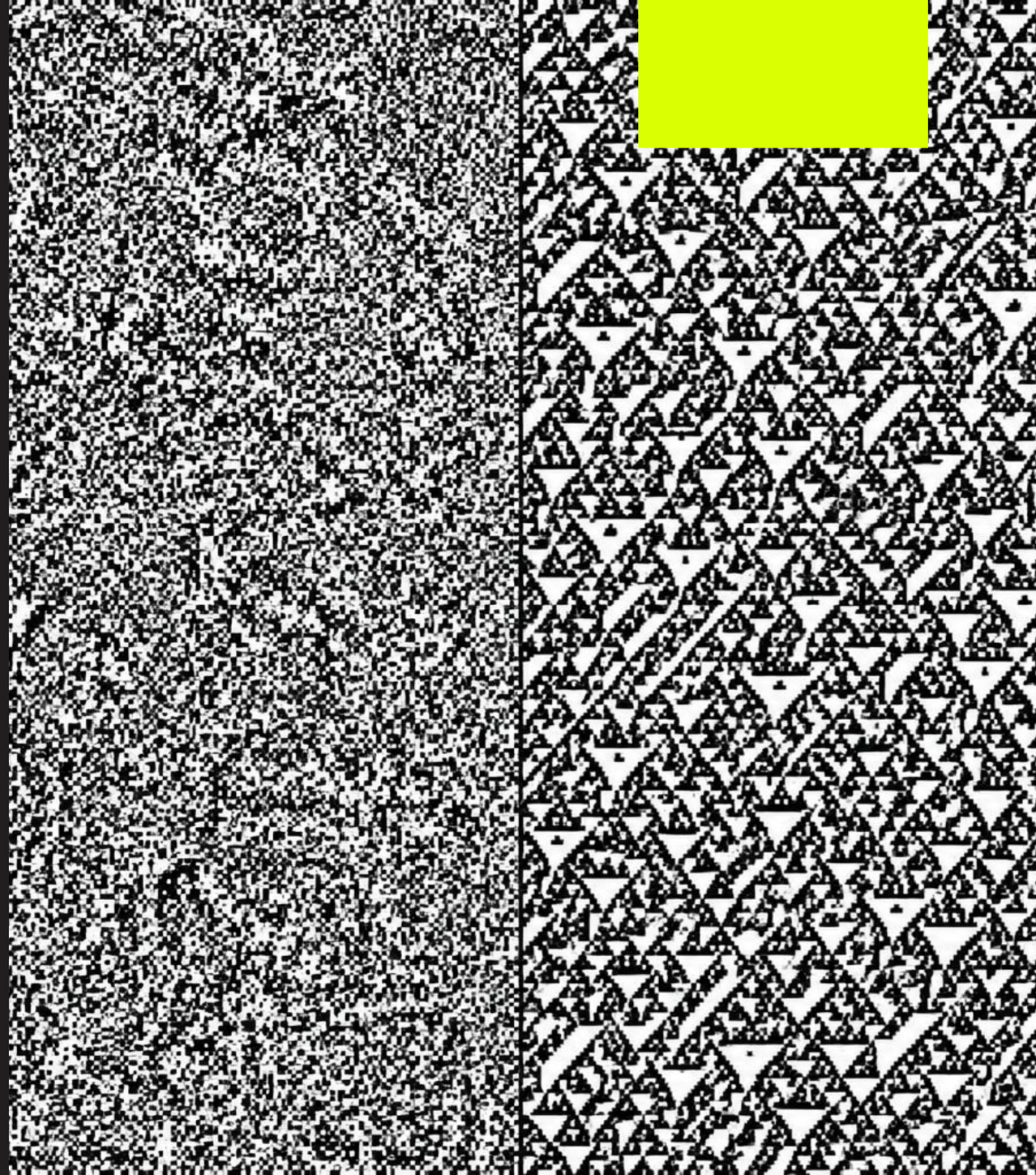

gzip
WHAT IS THE
COMPLEXITY?
512 random squares
REVERSED
copy of first 512

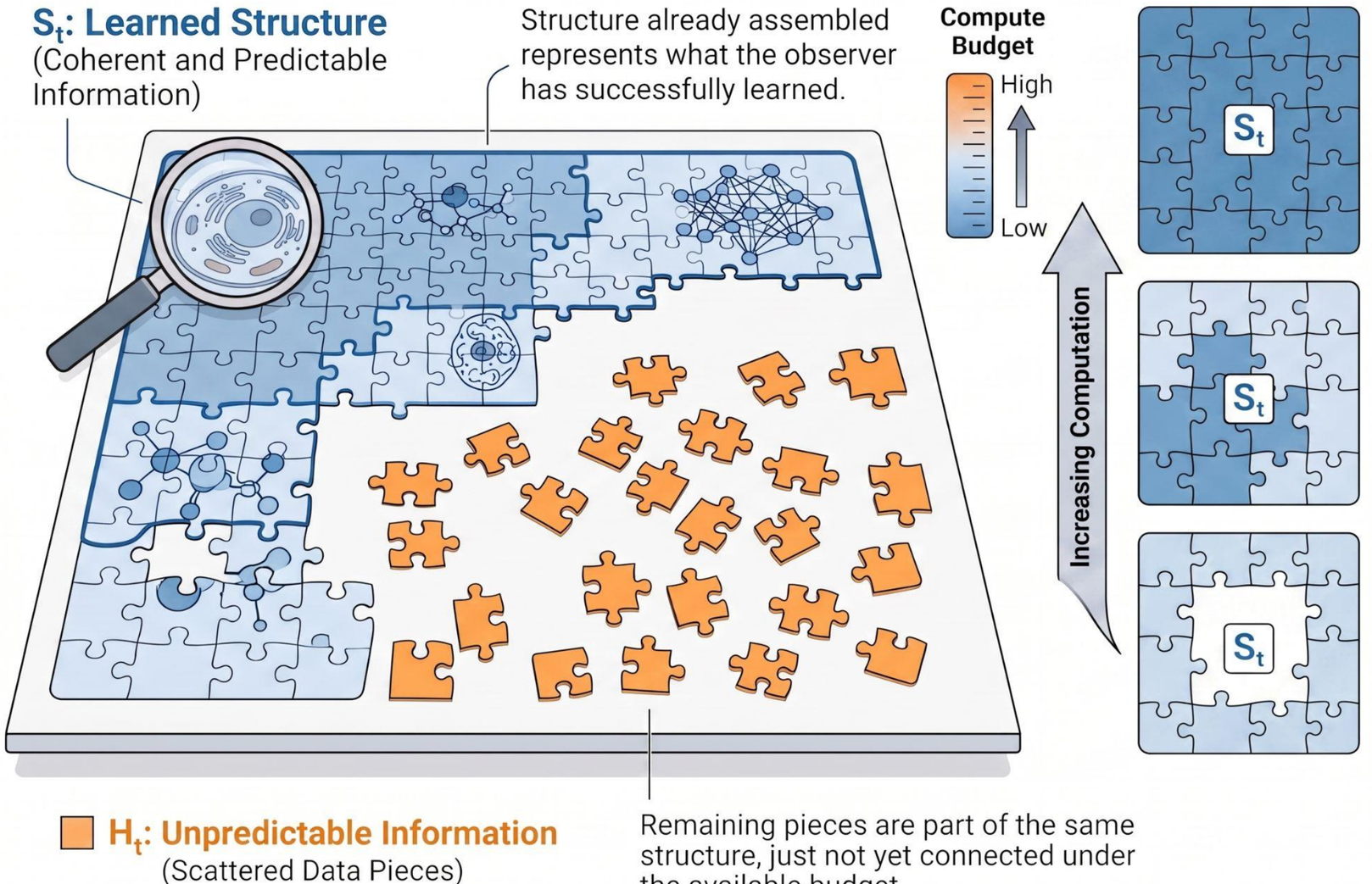
Complexity is observer-relative. But complexity is not the same as learnable structure.



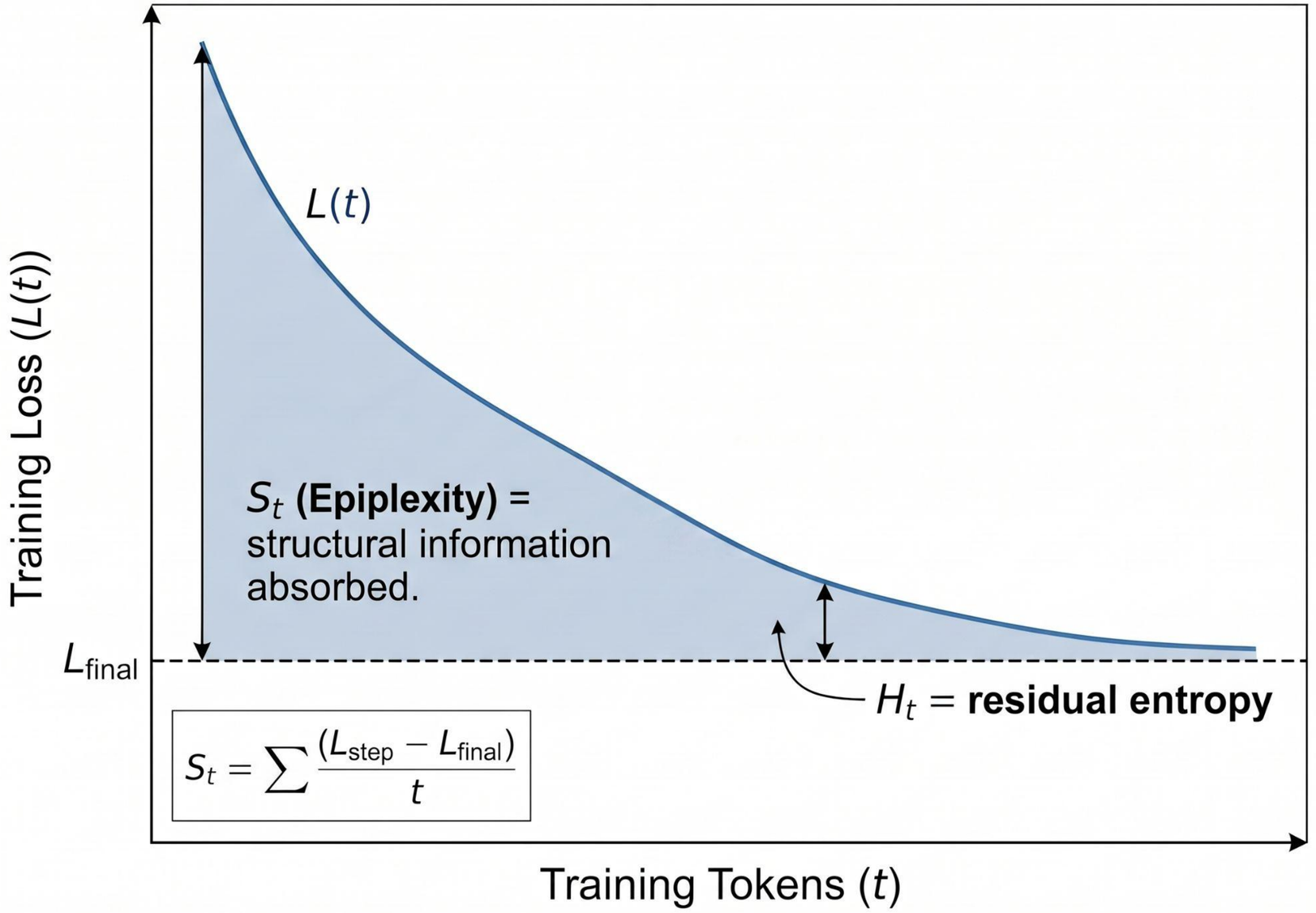

high
low
Entropy
Noise
(CSPRNG, Rule 30)
00118298350363784372523522
71306858384393930132356 4
63718967583085259017333 48
86993034370638918295829 81
18242209393838217842678 63
Natural systems
(language, chess, NCA)
Text text, and now
language in the
cartoar automatics
text take ards are
mown, chess from
chess eevit oment,
or cellular auto-
thess text.
Entropy and Structure are orthogonal axes:
High complexity does not always mean
learnable structure.
00000000
or ABAABABAB
Trivial (constant strin-
gs, repetitions)
$a^n b^n$
aaabbbccc
Counting Sequences
low
Structure
high
THE QUESTION: How do we separate raw randomness
from learnable, reusable structure?

Epiplexity (2601.03220) measures the structural, learnable information that a computationally bounded observer can extract from data.



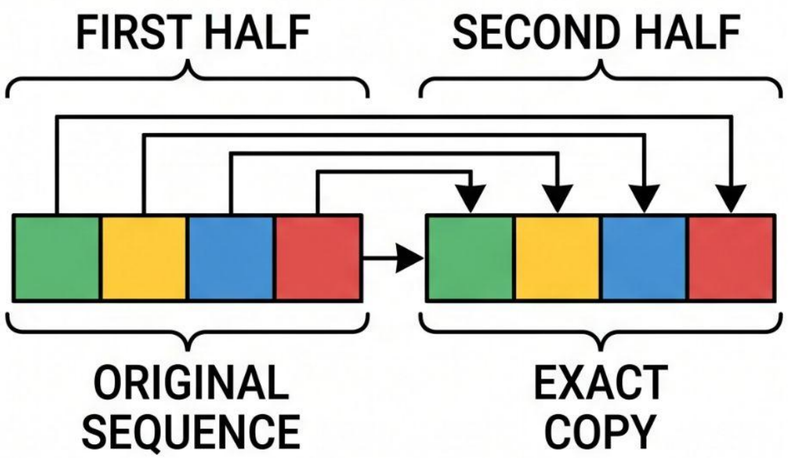

S_t: Learned Structure
(Coherent and Predictable Information)
Structure already assembled represents what the observer has successfully learned.
Compute Budget
High
Low
Increasing Computation
S_t
S_t
S_t
H_t: Unpredictable Information
(Scattered Data Pieces)
Remaining pieces are part of the same structure, just not yet connected under the available budget.

You already measure epiplexity — it's hiding in your loss curves.

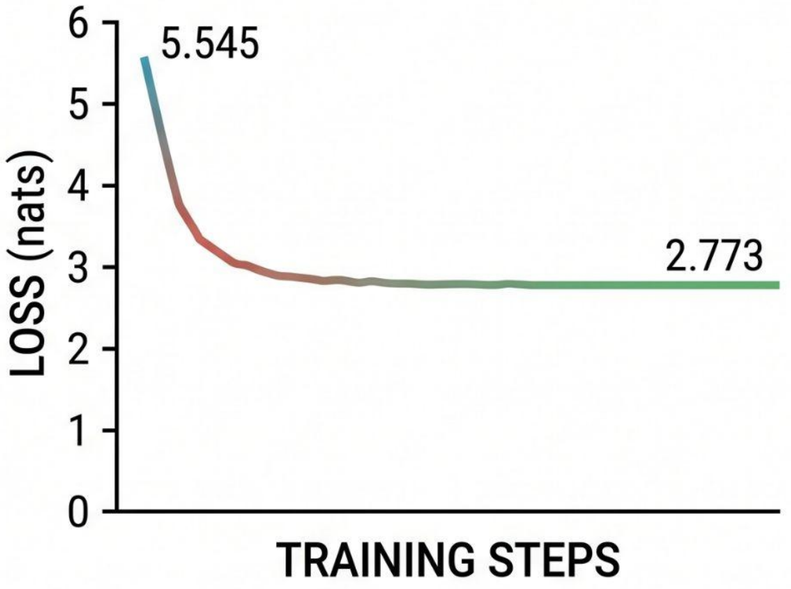


For a pattern where the second half is an exact copy of the first half, epiplexity tells a clear story: half the information is learnable structure.

PATTERN: COPY



TRAINING LOSS CURVE

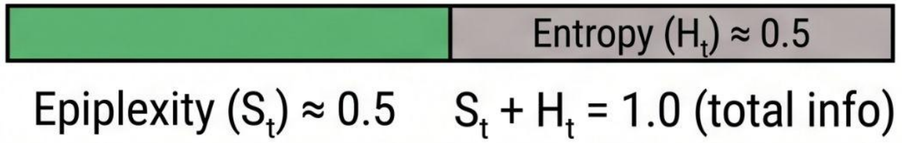


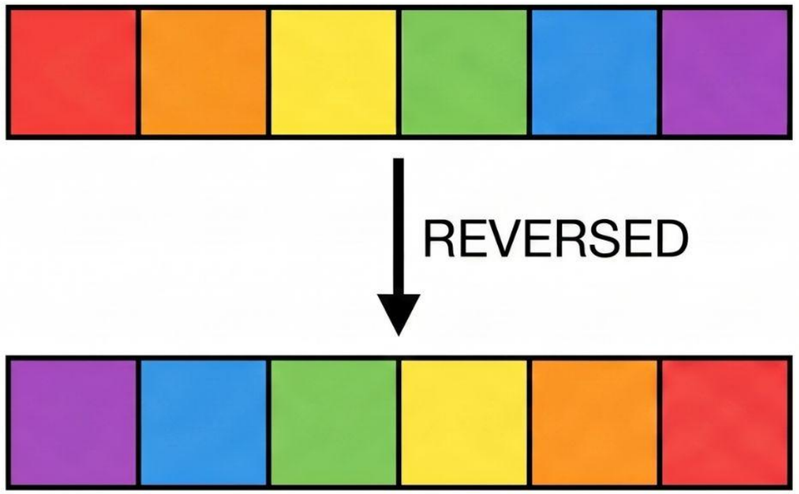
QUANTITATIVE ANALYSIS

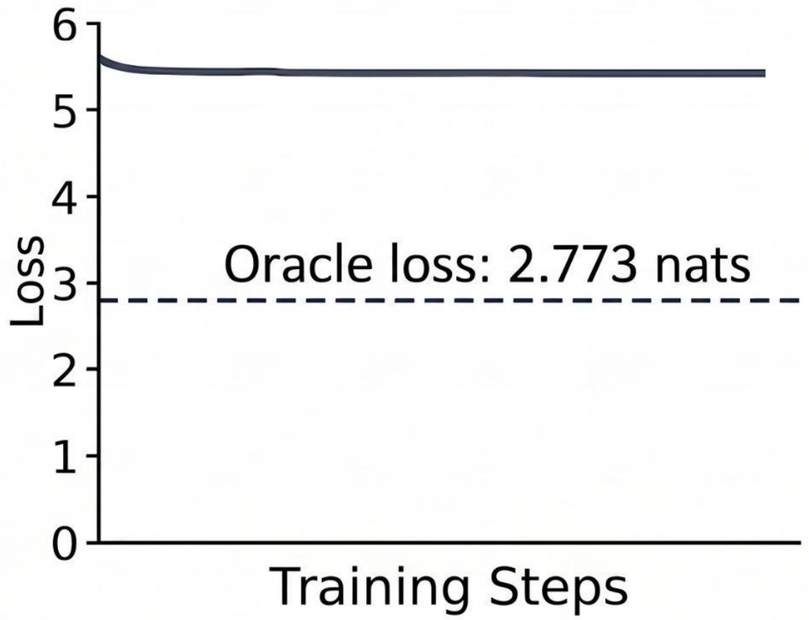
Vocabulary: 256 symbols

Uniform loss: ln(256) = 5.545 nats

Oracle loss: 2.773 nats (half)







Vocabulary: 256 symbols

Uniform loss: $\ln(256) \approx 5.545$ nats

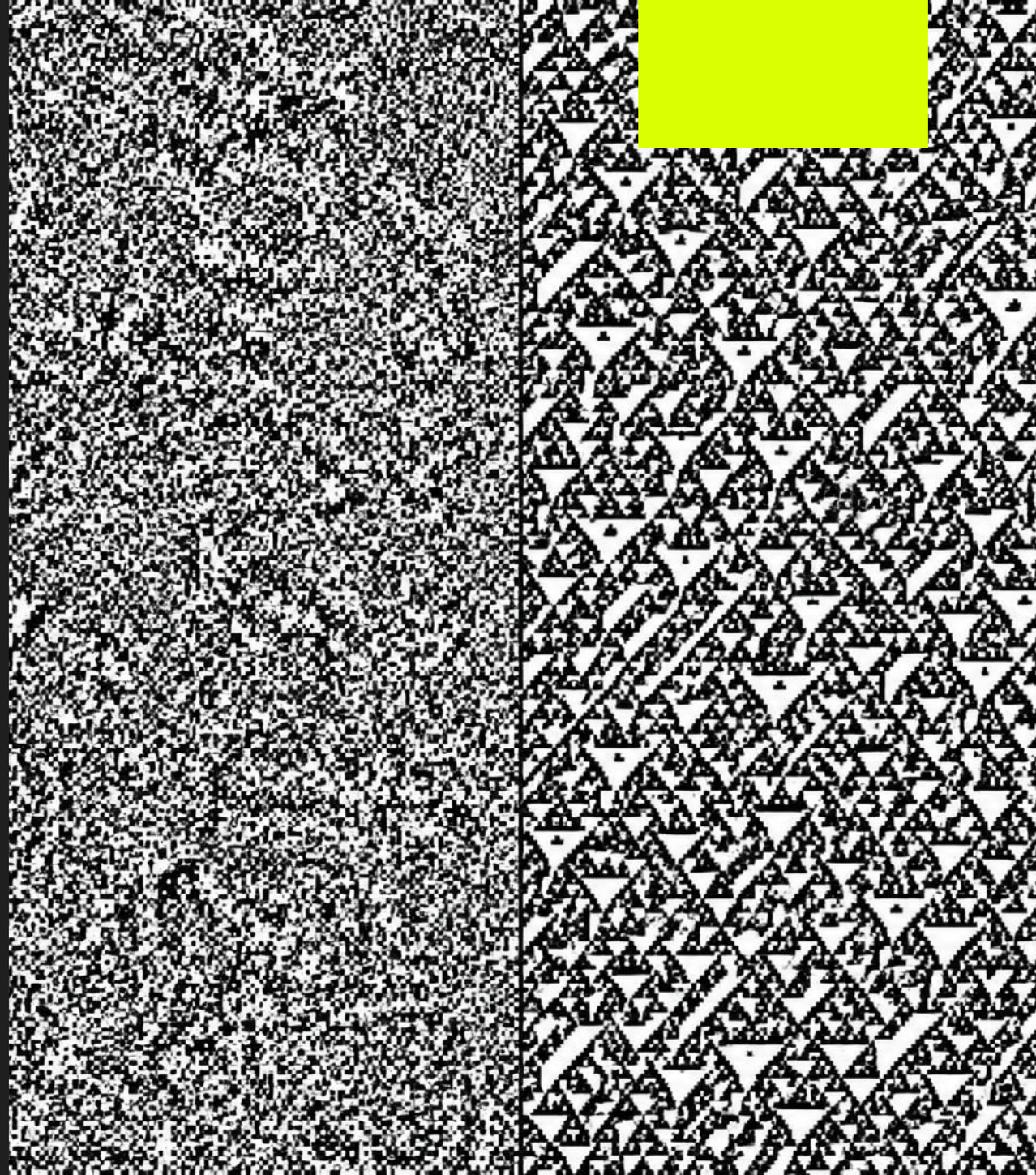
Oracle loss: 2.773 nats (half)

S_t (Epiplexity): ≈ 0.0

H_t (Entropy): ≈ 1.0 (all of it)

Epiplexity reveals the ARCHITECTURE'S inductive biases!

Because TRUE epiplexity is incomputable and observer-dependent, we need to be careful. We can't just say "this pattern has high epiplexity/structure" — we have to specify the observer and the compute budget. Regardless, these are our tools to speak about the *complexity* of data.



How do we run the experiments?

STEP 1: Pre-Pretraining

>> Train a model (decoder-Llama) on synthetic patterns (Dyck brackets, copies, counting, NCA...)

STEP 2: RESET

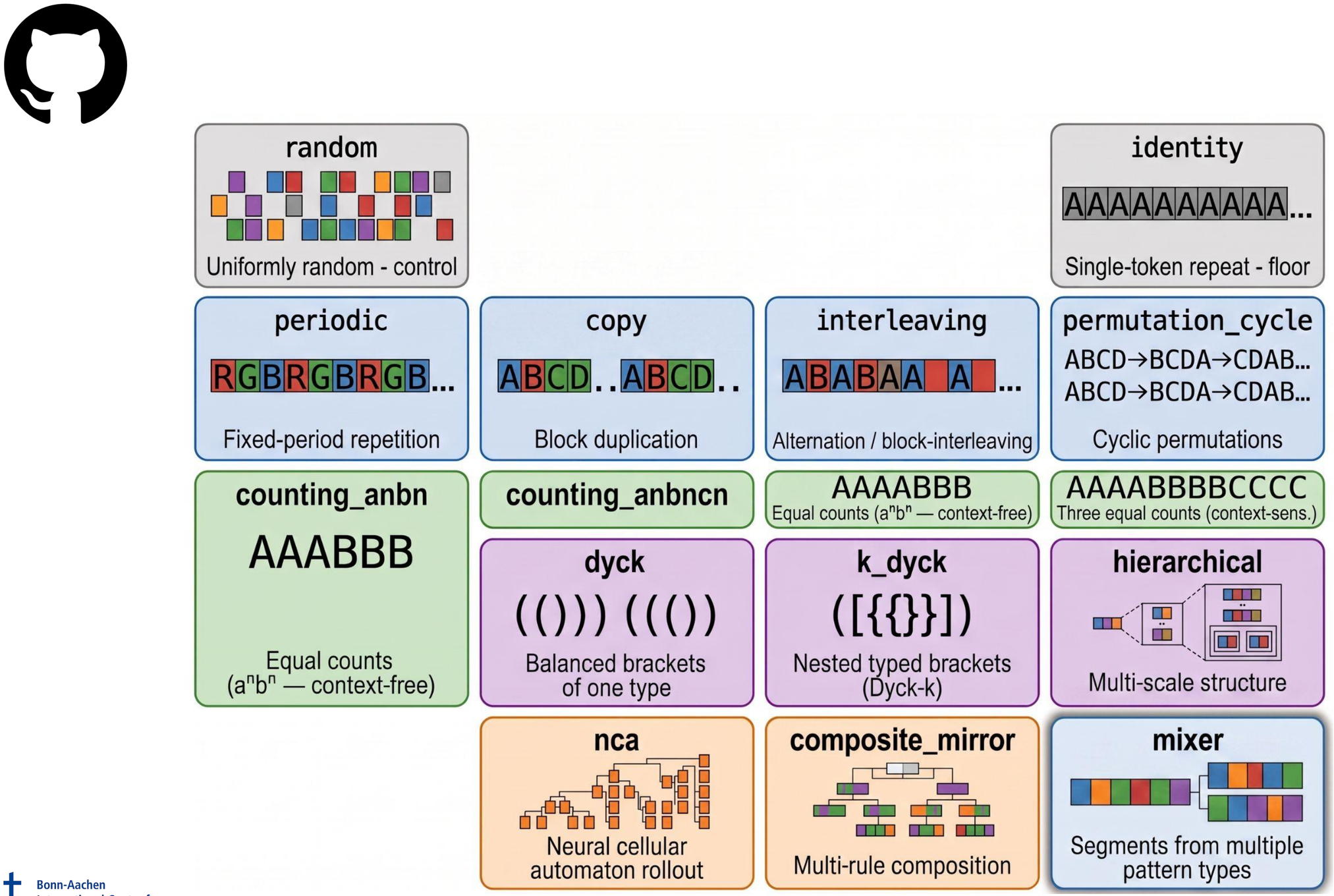
>> Keep attention weights, reinitialize embeddings.

STEP 3: Pretraining

>> Train ALL conditions on a fixed budget of real web text (e.g., c4)

STEP 3: Evals

>> Measure val. loss and downstream performance on canarie benchmarks.


random
Uniformly random - control
identity
AAAAAAAAAA...
Single-token repeat - floor
periodic
RGBRGBRGB...
Fixed-period repetition
copy
ABCD..ABCD..
Block duplication
interleaving
ABABAA A ...
Alternation / block-interleaving
permutation_cycle
ABCD→BCDA→CDAB...
ABCD→BCDA→CDAB...
Cyclic permutations
counting_anbn
AAABBB
Equal counts
(aⁿbⁿ — context-free)
counting_anbncn
AAAABBB
Equal counts (aⁿbⁿ — context-free)
AAAABBBBCCCC
Three equal counts (context-sens.)
dyck
(()))(((())
Balanced brackets
of one type
k_dyck
([{{}}])
Nested typed brackets
(Dyck-k)
hierarchical
Multi-scale structure
nca
Neural cellular
automaton rollout
composite_mirror
Multi-rule composition
mixer
Segments from multiple
pattern types

How do we run the experiments?

For ALL these patterns we “know”:

>> **Gzip complexity** : Raw compressibility; a cheap proxy for total information content.

>> **Oracle loss** : The theoretical minimum loss if the model knew the exact data-generating rule.

>> **S□ (epiplexity):** The bits the model spent learning the pattern; the area under the loss curve.

>> **H□ (entropy):** The bits of residual unpredictability after the model extracted all learnable structure.

Can we demonstrate that any of these measures correlate with—or even causally influence—changes in downstream performance?

What did we find?

The experiment was designed to answer a cascade of questions:

>> Q0: Is there a universal "sweet spot" of undertraining that predicts transfer?

>> Q1: Do gzip and epiplexity measure the same thing?

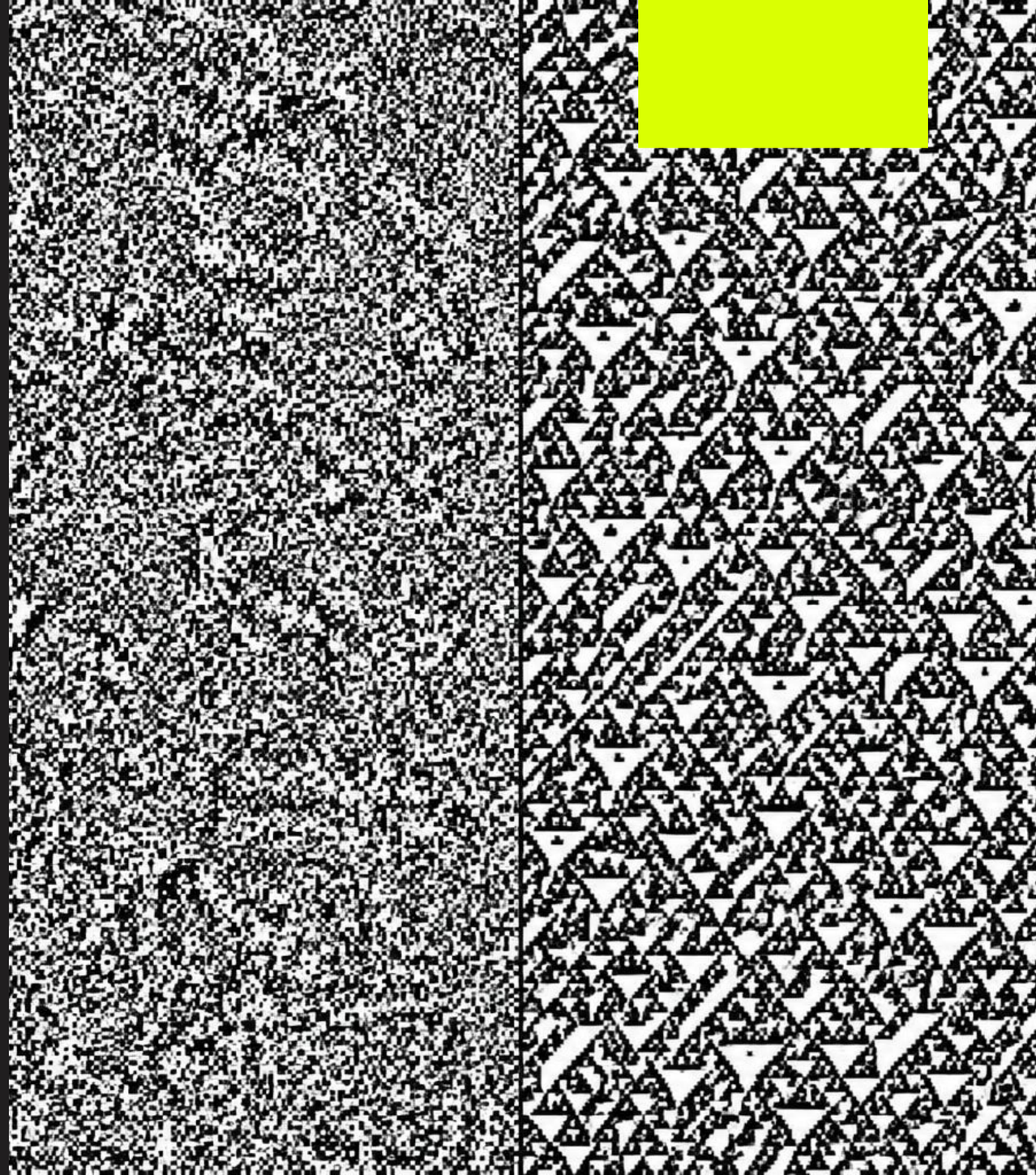
>> Q2: Does the complexity-matching hypothesis hold?

>> Q3: Can ANY Phase-1 measurement predict downstream transfer?

>> Q4: Is Phase-1 structural information (S□) conserved into Phase 2?

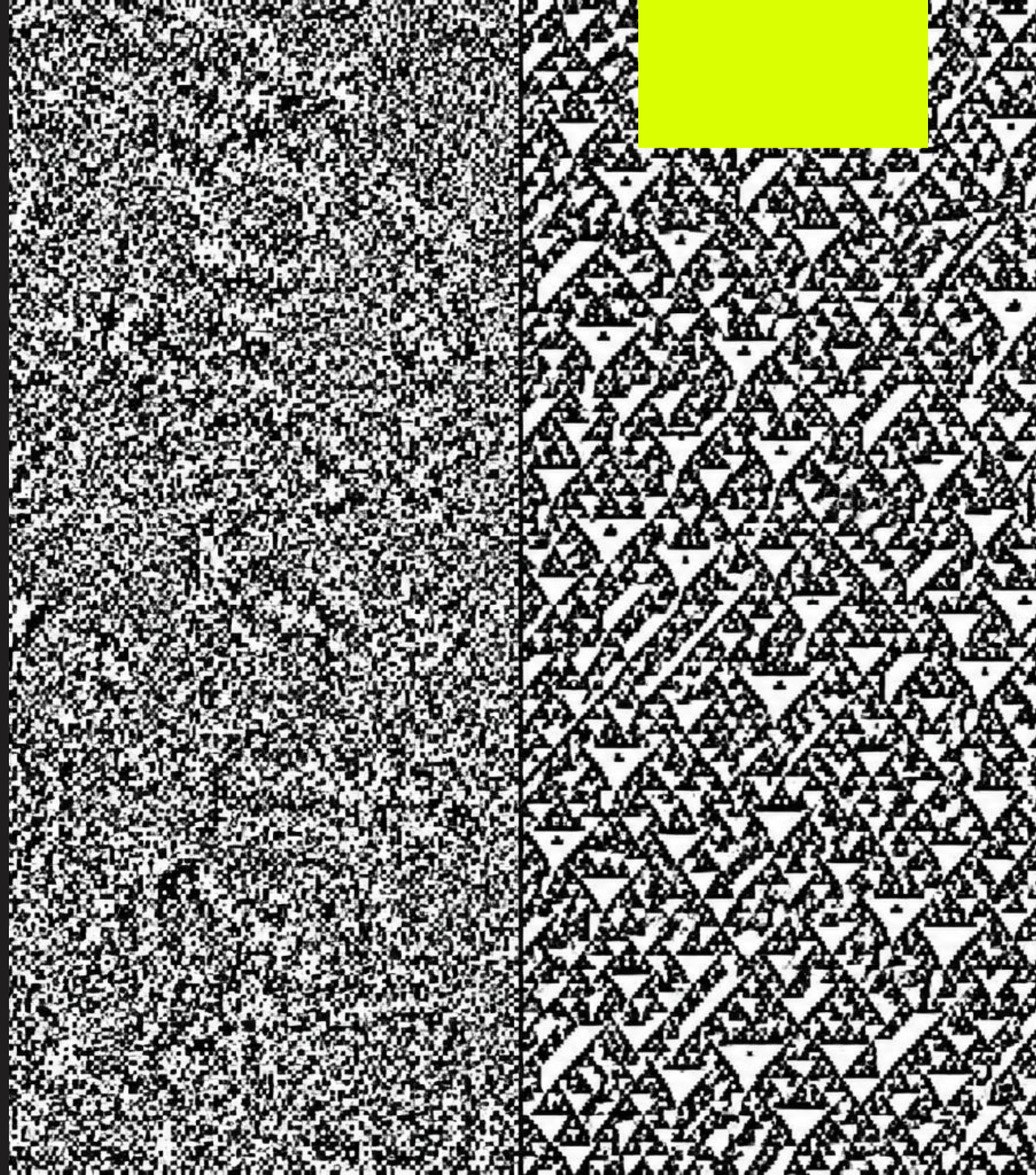
>> Q5: WHEN does the advantage appear during Phase 2 training?

What we know so far is that pre-pretraining on synthetic patterns yields small but consistent improvements, although many of the results are counterintuitive and contradict findings reported in parts of the literature (Find the Full Story).



Thank You

Nicholas Kluge Corrêa
Postdoc @ Bonn-Aachen International Center for Information Technology (b-it)



b-it Bonn-Aachen International Center for Information Technology

UNIVERSITÄT BONN